Exp. No: 6
Date: 8.6.25

Error Detection and Correction using Hamming Code.

Aim :-

To Write a program to implement error detection and correction using the Hamming Code.

Algorithm :-

1. Start the program
2. Read the number of 4-bit data blocks (n) forming the input data stream.
3. for each 4-bit data block $D = d_1 d_2 d_3 d_4$, Perform Hamming Code :

a) Arrange 7 bits with parity bits at position 1, 2, and 4 and data bits at position 3, 5, 6 and 7..

b) Compute the parity bits using even parity :

$P_1 = d_1$ XOR d_2 XOR d_4

$P_2 = d_2$ XOR d_3 XOR d_4

$P_4 = d_2$ XOR d_3 XOR d_4.

c) Form the 7 bit Hamming Code word : $P_1 P_2 d_1 P_4 d_2 d_3 d_4$

4) Simulate transmission by introduction a single bit error at a chosen bit position in code word

5) At the receiver, recompute the check bits from the received code word.

$C_1 = P_1$ XOR d_1 XOR d_2 XOR d_4

$C_2 = P_2$ XOR d_1 XOR d_3 XOR d_4

$C_4 = P_4$ XOR d_2 XOR d_3 XOR d_4

```
=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===

Enter number of 4-bit data blocks in the stream: 2

--- Block 1 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1
Original Data Bits : 1011
Encoded Hamming Code (7 bits): 0110011
Enter bit position to introduce error (1-7, 0 = no error): 5
Transmitted Code (with error induced at pos 5): 0110111
Received Code : 0110111
Checking for errors...
Error detected at bit position: 5 -> Correcting...
Corrected Code : 0110011
Recovered Data Bits : 1011

--- Block 2 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1
Original Data Bits : 1011
Encoded Hamming Code (7 bits): 0110011
Enter bit position to introduce error (1-7, 0 = no error): 0
Transmitted Code (no error induced): 0110011
Received Code : 0110011
Checking for errors...
No error detected.
Corrected Code : 0110011
Recovered Data Bits : 1011
```

6 6) Compute Error position = c4 c2 c1 .

7) If Error Position = 0 . no error has occurred.

Else, the bit at "Error Position" is inverted to correct the Error.

8) Extract the original 4 data bits from the Corrected 7 bit code word.

9) Repeat steps 3 to 8 for all n blocks in the data stream

10) Stop the program.

Program :-

```
def encode(d):
    h = [0] * 7
    h[2], h[4], h[5], h[6] = d
    h[0], h[1], h[3] = h[2] ^ h[4] ^ h[6], h[2] ^ h[5] ^ h[6], h[4] ^ h[5] ^ h[6]
    return h

def detect_and_Correct(h):
    e = (h[0] ^ h[2] ^ h[4] ^ h[6]) + 2* (h[1] ^ h[2] ^ h[5] ^ h[6])
                                    + 4* (h[3] ^ h[4] ^ h[5] ^ h[6])
if e:
    Print (f" Error detected at bit position : {e} -> Correcting ...")
    h[e-1] ^= 1
else:
    Print ("No error detected.")
```

```
def s(b):
    Return ''.join(map(str,b))
print("=== Hamming Code ===")
for bin range(1, int(input("Enter number of bits : "))+1):
    Print(f"\n--- Block {b}---")
    d = list(map(int,input("Enter {d} data bits (Space-Separated: ")
                                                    .split()))
    h = encode(d)
    print(f"Original Data Bits : {s(d)}")
    Print(f"Encoded Hamming Code : {s(h)}")
    p = int(input(f"Enter bit position : "))
    if 1 <= p <= 7:
        h[p-1] ^= 1
        print(f"Transmitted Code {p} : {s(h)}")
    else:
        Print(f"Transmitted Code [No Error]: {s(h)}")
    Print(f"Received Code : {s(h)}")
    Print("Checking for error...")
    detect_and_Correct(h)
    Print(f"Corrected Code : {s(h)}")
    Print(f"Recovered Data Bits : {s([h[2], h[4], h[5], h[6]])}")
```

Result :-

The program to implement error detection and correction using the Hamming Code Concept was successfully written in python, executed and verified using a sample input data stream.